MX Explained

Tom Cranstoun

MX Explained 3
- What MX Is 3
- Why MX Exists 4
- The Philosophy 5
- Defer, Never Duplicate 6
- Container Independence 7
- Neutrality 8
- Readable and Trustworthy 9
- When and Where MX Is Useful 10
- Evidence, Not a Compliance Grant 11
- Inspect This Document 12

MX Explained

What MX Is

MX is metadata that records a file's provenance, context, and intended use, and travels with the file.

It matters because it keeps content usable when an AI agent, or any other system, encounters the file outside the environment that produced it.

MX stands for Machine Experience. It is to machines what UX is to people. User experience asks whether a person can understand and navigate something. Machine experience asks whether a machine can understand it and act on it correctly. MX is a practice, not a product: a way of writing files so that the next reader, human or machine, does not have to guess.

Why MX Exists

Machines now decide what gets read, cited, recommended, and bought. They decide from what they can parse and what they can verify. When they can do neither, they guess.

A guess at scale is not a small error. It is a misquoted price, a wrong dosage, a policy that was never written, or a service someone could not reach, and nobody is accountable afterwards.

A better model does not fix this. A better record does: content that declares what it is, evidence that travels with the file, and a named person who stands behind it and can still be found. MX is the practice that produces that record.

The problem grows as files leave home. On its home site, a web page gets context from navigation, branding, an about section, and a contact form. A PDF downloaded from that site, a dataset copied into a pipeline, or an image pasted into a report loses all of it. Whatever the file needs a machine to know, it has to carry itself.

The Philosophy

Metadata Is a Protocol

In MX, metadata is not documentation about a file. It is an instruction to the next machine that reads it. A field that says a signature expires on a given date tells a machine to stop trusting the file once it passes. A field that declares a conflict of interest tells a machine to weigh that bias. A field that lists external sources tells a machine it may check them independently.

So every field is written with a question in mind: what should the next reader check, verify, or refuse? We call this Metadata As The Engine Of Application State, or MATEOS.

Design for Both

Every file serves two audiences at once: the person who reads the prose and the machine that reads the structure. Both are first-class. The visible text is for eyes; the metadata behind it is for the agent, the screen reader, the validator, and the parser. Both say the same thing.

Documents Are Self-Sufficient

A file removed from the website that published it must still make sense to any machine that finds it. That rules out context that lives only in the surrounding page, and it rules in metadata embedded in the file itself.

Design for the Worst Machine

You cannot tell which machine is reading. It might be a large hosted model, a small local one with a tiny context window, a browser extension, or an industrial controller. MX designs for the least capable reader: critical facts stated explicitly, structure declared rather than implied, nothing locked behind a script. If the weakest machine understands the file, every other reader will too.

Trust Is Structural

A file can be perfectly structured and still give an agent no reason to trust it. Who published it? Has it changed in transit? Is this the current version? Self-declared answers prove nothing, so trust has to come from structure outside the file: a signed, dated, independently checkable record.

Corroboration over Assertion

A signature proves who published a claim, not that it is right. When a machine has to choose between options, it gives more weight to a claim it can ground in an authoritative external source than to one the publisher only asserts about itself. So a claim that others will act on points to an independent source that states the same thing, and where sources disagree, the file says so instead of hiding it.

Defer, Never Duplicate

MX does not invent what already exists. Where an established standard covers a need, MX uses that standard's own fields and adds nothing of its own. Deferring is not giving up responsibility; it is refusing to create a second, competing answer to a question that already has one.

The standards MX defers to include:

Concern	Standard MX Uses
Resource identity (date, format, rights, language)	Dublin Core
Web content, people, organisations, authorship	Schema.org
Datasets and data catalogues	DCAT
Tabular data and CSV columns	CSVW
Embedded media metadata	EXIF, IPTC, XMP, ID3
Usage permissions and duties	ODRL
Content authenticity for any medium, text included	C2PA Content Credentials
General provenance graphs	W3C PROV-O
API descriptions	OpenAPI
Accessibility	WCAG and ARIA
Language tags, currency codes, dates	BCP 47, ISO 4217, ISO 8601

A file describing a dataset, an image, an API, or a package uses those vocabularies directly. MX adds only the governance layer they leave out: how AI agents may use the content, where it came from and who answers for it, and how a machine should route and verify it.

Take content authenticity as an example. C2PA seals the bytes of a file so that tampering shows. MX makes the claims about that file readable by a machine. A registry then adds the signals a single embedded manifest cannot carry: currency (is this still the current version?) and freshness (does the publisher still stand behind it?). Each layer does one job, and none repeats another.

A well-built MX page therefore meets the standards of search engines and accessibility too. MX builds on established practice. It does not replace it.

Container Independence

MX metadata does not belong to any one file format. The container that holds MX metadata is called a COG, a Community Owned Governance Standard record, and a COG is carrier-neutral: the same fields travel in whatever holds the content.

Carrier	Where the MX Metadata Lives
Markdown	YAML frontmatter at the top of the file
HTML	`<meta name="mx:...">` tags and a source-frontmatter block in the page head
PDF	An XMP packet embedded in the file
Shell scripts	A commented YAML header
JavaScript and CSS	Comment blocks with `@mx:` tags
JSON	A root `mx` object
Databases and logs	A row or a structured log line

A field keeps one name in every carrier. `contentPolicy` in Markdown frontmatter is `mx:contentPolicy` in an HTML meta tag and in a PDF's XMP packet. A reader converts nothing, so the served page and its source agree by construction.

This document is a worked example. It was written as Markdown with MX frontmatter. The PDF you are reading was generated from that Markdown, and the same fields were written into its XMP packet, along with a fingerprint of the exact source used to build it. Its provenance travels beside it as a pair of records, one for the automated steps and one for the steps a person or a model took, including the human check. Separate the PDF from this website and it still says what it is, who wrote it, and where it originated.

Neutrality

MX is neutral in every direction that matters to someone deciding whether to adopt it.

Machine-neutral. There is no single AI whose preferences you can target. Different assistants fetch pages differently, weigh different signals, and change their behaviour whenever a vendor updates a model or a system prompt. A tactic that earns citations from one can be ignored by another. MX invests in the layer underneath: correct structure, declared metadata, and content that agrees with itself. That layer serves every reader, including the ones that have not been built yet.

Platform-neutral. MX does not care which content management system, framework, or hosting provider you use. It is metadata in files, and every platform produces files.

Model-neutral. Because the governance travels in the files rather than in any model, agent framework, or prompt, you bring your own and can replace any of them without losing it.

Vendor-neutral. The MX standard is open, MIT-licensed, and governed by The Gathering, an independent, community-led standards body run on the same model as the W3C, so no single vendor can capture it. You can adopt the standard without adopting any vendor, including us.

Readable and Trustworthy

MX is one layer in a larger structure. Each layer has one job:

- **MX** makes files understandable to machines: the metadata and the practice of writing it.
- **The Gathering** decides what that metadata is. It fills the gaps existing standards leave and never reinvents what they cover.
- **COGs** are the carrier-neutral containers that hold the metadata.
- **A registry** is public infrastructure that holds, indexes, and serves COGs, so machines can discover them and check them.
- **REGINALD** (Registry for Genuine Information, Notarised Authentication, and Legitimate Documentation) is CogNovaMX's registry, built on the open standard.

The first four layers are open and governed by The Gathering. REGINALD is proprietary. Other registries can run on the same standard, and REGINALD's job is to be the best-operated one.

The two products work as a pair. **MX makes content machine-readable. REGINALD makes it machine-trustworthy.** Each is incomplete without the other: readable but unverified content is a well-formatted guess; verified but unreadable, it is a locked box.

Trustworthy is not a single yes or no. REGINALD gives a machine separate signals, each answering a question it can check on its own:

- **Attestation:** who published this, and when? A signed, dated record, verifiable by the machine itself.
- **Currency:** is this the current version? New versions supersede old ones, and old ones stay queryable instead of vanishing.
- **Freshness:** is the publisher still standing behind this? A liveness signal that holds while the publisher stays active and freezes when they go quiet.

Corroboration completes the picture, and it is carried by the content itself rather than the registry. Readable, attributable, corroborated: a claim that survives a machine's summarising, names someone who answers for it, and cites a source better than the publisher's own word.

REGINALD is in alpha.

When and Where MX Is Useful

MX earns its place wherever a machine will meet your content without you there to explain it:

- **Files that travel.** PDFs, datasets, images, spreadsheets, and slide decks that are downloaded, forwarded, or ingested far from the site that published them.
- **Sites that agents read and act on.** Product catalogues, prices, policies, opening hours, and services that an assistant may quote, compare, or book on someone's behalf.
- **Content that must stand up to scrutiny.** Where an organisation may be asked to show what it published, when, and who approved it, MX turns that evidence into structured, machine-checkable records.
- **Estates that use more than one AI.** When several models, agents, or vendors touch the same content, metadata in the files is the one layer they all share.
- **Accessibility work.** The explicit structure a machine needs is exactly what a screen reader needs.

MX adds little to content that no machine will ever read, or to throwaway material with no audience beyond the moment. The effort pays back in proportion to how far the file travels and how much rides on a machine reading it correctly.

Evidence, Not a Compliance Grant

> **Note:** This page describes regulatory frameworks in general terms only. Nothing here is legal advice. Requirements vary by jurisdiction, organisation type, and use case. Consult qualified legal specialists for guidance specific to your situation.

Rules on how AI systems use and produce content are emerging in many jurisdictions. MX and REGINALD do not make an organisation compliant with any of them; compliance stays its own legal duty. What they provide is evidence: structured, machine-readable, tamper-evident records an auditor, an operator, or a regulator can query and verify with standard tools. Sold as evidence, that is what those readers need. Sold as compliance in a box, it would collapse on the first inquiry.

Inspect This Document

Scan the QR code on the last page, or visit mx.allabout.network/tell. That page lets you download this PDF or open it in the MX Inspector, where you can see the metadata embedded in it, the fingerprint of the source it was built from, and the provenance records that travel beside it. The inspection runs entirely in your browser.

That is MX in practice: a file that can tell a machine what it is.

MX

MACHINE EXPERIENCE

Compatible

tagged · provenance · attested

Scan to learn more

This PDF is MX Compatible. It carries machine-readable structure, AI-governance provenance, and the metadata that lets a regulator, an AI agent, or a screen reader walk the document with the same confidence a sighted reader does. Scan the QR code or visit `https://mx.allabout.network/tell` to read what that means.

Click here to Inspect this document's MX Metadata

One click fetches this document and runs the full inspection in your browser: the verdict, every MX metadata field, and the provenance walk.

Tagged PDF · Provenance: EU AI Act · UK ICO · NIST AI RMF